OS PROJECT REPORT

PHASE-1

1.A

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ make clean
  make test_objects
  ./test_objects

rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o
test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c
4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$
```

1.B

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ find .pes/objects -type f

.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

PHASE-2

2.A

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ make test_tree
  ./test_tree

gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$
```

2.B

PHASE-3

3.A

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ make pes
  rm -rf .pes
  ./pes init
  echo "hello" > file1.txt
  echo "world" > file2.txt
  ./pes add file1.txt file2.txt
  ./pes status

gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt
```

```
Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_tree.c
  untracked:  commit.c
  untracked:  commit.h
  untracked:  .gitignore
  untracked:  pes.c
  untracked:  object.c
  untracked:  Makefile
  untracked:  README.md
  untracked:  pes.h
  untracked:  test_tree
  untracked:  tree.c
  untracked:  a.txt
  untracked:  index.c
  untracked:  index.h
  untracked:  tree.h
  untracked:  test_sequence.sh
  untracked:  test_objects
  untracked:  test_objects.c
  untracked:  sample.txt
```

3.B

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ cat .pes/index

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776361924 6 fi
le1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776361924 6 fi
le2.txt
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$
```

PHASE-4

4.A

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ rm -rf .pes
  ./pes init
  echo "Hello" > hello.txt
  ./pes add hello.txt
  ./pes commit -m "Initial commit"

  echo "World" >> hello.txt
  ./pes add hello.txt
  ./pes commit -m "Add world"

  echo "Goodbye" > bye.txt
  ./pes add bye.txt
  ./pes commit -m "Add farewell"

  ./pes log

Initialized empty PES repository in .pes/
Committed: 7aa91c114fe5... Initial commit
Committed: 50faeac3a0a6... Add world
Committed: 31a388d9d2be... Add farewell
commit 31a388d9d2be92132d09033e5ce8cea72bec3226fcc267935f695d82fcf2adea
Author: PES User <pes@localhost>
Date:   1776362008

    Add farewell

commit 50faeac3a0a6fe7e1d0bce7ec789a02fb830b5c5e8be941e4d4169fe2fcf4f55
Author: PES User <pes@localhost>
Date:   1776362008

    Add world

commit 7aa91c114fe566404f853c9e68cf152216de6b42ff84bb109def97157afcb64e
Author: PES User <pes@localhost>
Date:   1776362008
```

4.B

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$  find .pes -type f | sort

.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/31/a388d9d2be92132d09033e5ce8cea72bec3226fcc267935f695d82fcf2adea
.pes/objects/50/faeac3a0a6fe7e1d0bce7ec789a02fb830b5c5e8be941e4d4169fe2fcf4f55
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/7a/a91c114fe566404f853c9e68cf152216de6b42ff84bb109def97157afcb64e
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$
```

4.C

```
ditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ cat .pes/refs/heads/main
 cat .pes/HEAD

31a388d9d2be92132d09033e5ce8cea72bec3226fcc267935f695d82fcf2adea
ref: refs/heads/main
ditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$
```

FULL INTEGRATION

```
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$ make test-integration

=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
--- First Commit ---
Committed: 44425d53f8e3... Initial commit

Log after first commit:
commit 44425d53f8e326af66ae363fc9047178aeec41ed9fcf404cac60afc7aad0f71f
Author: PES User <pes@localhost>
Date:   1776362096

    Initial commit


--- Second Commit ---
Committed: 045d673f8097... Update file.txt

--- Third Commit ---
Committed: c4b9ce391630... Add farewell

--- Full History ---
commit c4b9ce3916303c5f192201223464cd26c5945b9595d5ba7796699aba5e5feaa4
Author: PES User <pes@localhost>
Date:   1776362096

    Add farewell

commit 045d673f809772a24b9d6dcee78f3c0960e31f4f46636482d466a5bdf8cea0a5
Author: PES User <pes@localhost>
Date:   1776362096

    Update file.txt

commit 44425d53f8e326af66ae363fc9047178aeec41ed9fcf404cac60afc7aad0f71f
Author: PES User <pes@localhost>
Date:   1776362096

    Initial commit
```

```
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c4b9ce3916303c5f192201223464cd26c5945b9595d5ba7796699aba5e5feaa4

--- Object Store ---
Objects created:
10
.pes/objects/04/5d673f809772a24b9d6dcee78f3c0960e31f4f46636482d466a5bdf8cea0a5
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/44/425d53f8e326af66ae363fc9047178aeec41ed9fcf404cac60afc7aad0f71f
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c4/b9ce3916303c5f192201223464cd26c5945b9595d5ba7796699aba5e5feaa4
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
aditya@aditya-VirtualBox:~/pes2ug24cs034-pes-vcs$
```

PHASE-5

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

ANS)

pes checkout <branch> would first read .pes/refs/heads/<branch> to get the commit hash of that branch. Then it would update .pes/HEAD so that HEAD points to that branch instead of the current one.

After that, the working directory must be changed to look exactly like the files stored in that branch's latest commit. That means creating files, deleting files that should not be there, restoring folders, and writing the correct contents from the stored objects. The index should also be updated to match the checked-out branch.

This becomes complex because checkout is not only switching a branch name. It must carefully update real files on disk, handle folders and file permissions, and make sure the user's uncommitted changes are not accidentally lost.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANS)

To detect a dirty working directory conflict, I would compare the current file on disk with the version recorded in the index. The index tells us what file is currently tracked and what hash it had when it was last staged. If the file's current contents no longer match that staged hash, then the file has uncommitted changes.

Next, I would compare the current branch's version of that file with the target branch's version by reading the trees and blobs from the object store. If the file is different between the two branches, and the working file is also different from the index, then checkout should refuse.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANS)

In detached HEAD state, commits are created normally, but they are not saved on any branch. HEAD points directly to the commit hash, so the commits can become unreachable later. The user can recover them by using the commit hash and creating a new branch that points to that commit.

PHASE-6

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANS)
Start from all branch heads, mark those commits as reachable, then recursively follow parent commits, trees, subtrees, and blobs. After that, scan .pes/objects/ and delete anything not marked reachable.

A hash set is the best data structure for tracking reachable hashes efficiently.

For 100,000 commits and 50 branches, you would still visit about 100,000 unique commits if branches share history, plus the reachable trees and blobs, so overall a few hundred thousand objects.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANS)

It is dangerous because GC may think a newly created object is unused and delete it before the commit is fully linked to a branch.

Example: while pes commit is running, it may first write a blob or tree object, but it has not yet written the final commit or updated HEAD. If GC runs at that moment, it may not see any branch pointing to that new object, so it may delete it. Then the commit finishes and ends up pointing to an object that is missing.

Git avoids this by using careful locking and by not deleting objects that are too new immediately. Real Git also uses reachability checks together with safety rules so objects being created during active operations are not collected too early.